

1. A matrix in Python

A matrix is a list containing other lists. Each inner list is a row.

```
matrix = [ [1, 2, 3], [2, 4, 6], [3, 6, 9], [4, 8, 12] ]
```

For a simple visual understanding, we write the same matrix (list of lists on multiple lines)

```
matrix = [  
    [1, 2, 3],  
    [2, 4, 6],  
    [3, 6, 9],  
    [4, 8, 12],  
]
```

This matrix has four rows and three columns. We can access one element using two indices:

```
print(matrix[0][0])  
print(matrix[2][1])
```

The first index selects the row, and the second index selects the column.

2. Print a matrix using row and value

The simplest way to visit every element is to loop through each row and then each value in that row:

```
for row in matrix:  
    for value in row:  
        print(value, end=" ")  
    print()
```

Expected output:

```
1 2 3  
2 4 6  
3 6 9  
4 8 12
```

3. Print a matrix using `i` and `j`

The same operation can be written with row and column indices. This is useful when we need to know the position of an element.

```
for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        print(f"matrix[{i}][{j}] = {matrix[i][j]}")
```

Here, `i` is the row index and `j` is the column index.

Exercise 1

Modify the loop so that it prints only the elements on the main diagonal. The condition for a diagonal element is:

```
i == j
```

4. Calculate row totals

Use a nested loop to calculate the total of every row:

```
for i in range(len(matrix)):
    row_total = 0

    for j in range(len(matrix[i])):
        row_total += matrix[i][j]

    print(f"Row {i}: {row_total}")
```

Expected output:

```
Row 0: 6
Row 1: 12
Row 2: 18
Row 3: 24
```

Matrix addition and transpose

Matrix addition

Two matrices can be added when they have the same number of rows and columns. Add the elements at the same positions:

```
A = [  
    [1, 2, 3],  
    [4, 5, 6],  
]  
  
B = [  
    [10, 20, 30],  
    [40, 50, 60],  
]  
  
C = []  
  
for i in range(len(A)):  
    row = []  
  
    for j in range(len(A[i])):  
        row.append(A[i][j] + B[i][j])  
  
    C.append(row)  
  
print(C)
```

Output:

```
[[11, 22, 33], [44, 55, 66]]
```

The indices `i` and `j` refer to the same position in all three matrices: `A[i][j]`, `B[i][j]`, and `C[i][j]`.

Exercise: matrix addition

Write a function `add_matrices(A, B)` that returns the sum of two matrices. Assume that the matrices have the same dimensions.

Matrix transpose

The transpose changes rows into columns. A 2x3 matrix becomes a 3x2 matrix:

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
]  
  
transpose = []  
  
for j in range(len(matrix[0])):  
    row = []  
  
    for i in range(len(matrix)):
```

```
        row.append(matrix[i][j])

    transpose.append(row)

print(transpose)
```

or

```
M_transpose = [ [ '' for _ in range(len(matrix)) ] for _ in range(len(matrix[0])) ]

for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        M_transpose[j][i] = matrix[i][j]
```

Output:

```
[[1, 4], [2, 5], [3, 6]]
```

Notice that the loop order is reversed: the outer loop visits columns, and the inner loop visits rows.

Exercise: transpose

Write a function `transpose_matrix(matrix)` that returns the transpose of a non-empty matrix. Test it with a 4x3 matrix.

5. Matrix values as strings

Data often comes from a file or user input, so values may initially be strings. Some values may be invalid:

```
scores = [
    ["12", "15", "missing"],
    ["8", "10", "9"],
    ["18", "absent", "13"],
]
```

Calling `int("missing")` raises a `ValueError`. Use `try` and `except` to handle the invalid value and continue:

```
for row in scores:
    for value in row:
        try:
            score = int(value)
            print(score)
```

```
except ValueError:
    print(f"Skipping invalid score: {value}")
```

Exercise 2

Write a function `total_valid_scores(scores)` that returns the total of all valid scores. Invalid values should be skipped. Test it with the matrix above.

6. A generator for valid scores

A generator produces values one at a time. The `yield` statement pauses the function and continues from the same place the next time a value is requested.

```
def valid_scores(matrix):
    for row in matrix:
        for value in row:
            try:
                yield int(value)
            except ValueError:
                print(f"Skipping invalid score: {value}")
```

Use the generator in a `for` loop:

```
for score in valid_scores(scores):
    print(score)
```

The generator does not create a separate list of valid scores.

Exercise 3

Write a function `score_summary(scores)` that uses `valid_scores(scores)` and returns:

```
(number_of_valid_scores, total, average)
```

Return `(0, 0, 0)` when there are no valid scores.

7. A generator that yields rows

Instead of yielding one score at a time, a generator can yield one complete row at a time:

```
def score_rows(matrix):
    for row in matrix:
        valid_row = []
```

```
    for value in row:
        try:
            valid_row.append(int(value))
        except ValueError:
            valid_row.append(0)

    yield valid_row
```

The value 0 represents a missing score in this version.

We can use the row generator to calculate each student's total:

```
for row in score_rows(scores):
    print(sum(row))
```

8. Control flow and data flow

The nested loops determine the order in which values are visited. This is the control flow.

The values move through the program from the matrix, through conversion and exception handling, to the generator and finally to the summary. This is the data flow.

Discuss:

1. Which loop visits each matrix element?
2. What happens when `int(value)` fails?
3. When does a generator execute its body?
4. Why is a generator useful when the matrix is very large?

Final exercise

Implement `score_summary(scores)` and test it with:

```
scores = [
    ["18", "16", "20", "17"],
    ["14", "absent", "15", "13"],
    ["20", "19", "18", "missing"],
    ["11", "12", "10", "14"],
]
```

The function should report 14 valid scores, a total of 217, and an average of 15.5. We use `ij` loops to print, as well as input data.